

20 AWS PRODUCTION ARCHITECTURE PROJECTS

| FOR SENIOR ENGINEERS |

- ✓ Real-World Architectures
- ✓ Production Use Cases
- ✓ Design Decisions
- ✓ Failure Scenarios
- ✓ Troubleshooting
- ✓ Best Practices



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



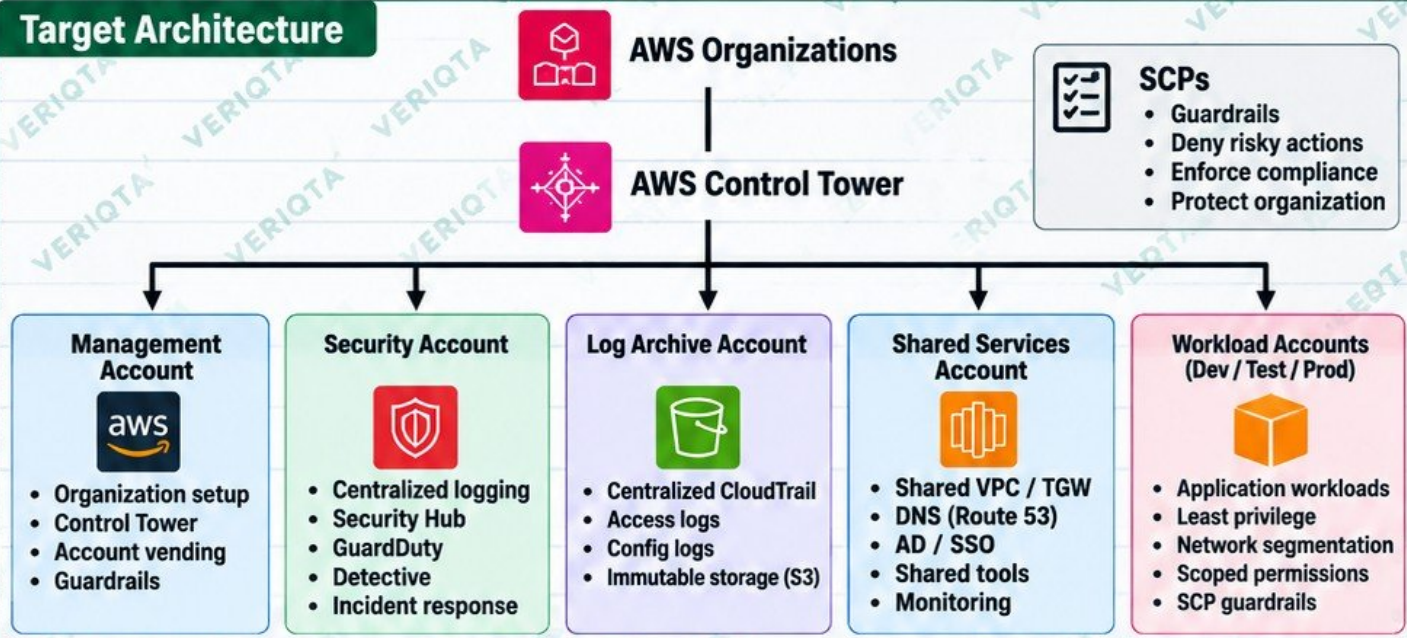
@veriqta_academy

Objective

Build an enterprise foundation with AWS Organizations, Control Tower, SCPs, centralized CloudTrail, security accounts, and account vending.

Senior Challenge: Contain a compromised workload account without disrupting shared organizational services.

Target Architecture



Key AWS Services

Service	Purpose
AWS Organizations	Multi-account management
AWS Control Tower	Landing zone and guardrails
Service Control Policies (SCPs)	Restrict or allow actions
AWS CloudTrail	Centralized audit logging
AWS Config	Resource compliance tracking
AWS Security Hub	Security findings
Amazon GuardDuty	Threat detection
AWS Identity Center	Centralized access management
Amazon S3 (Log Archive)	Immutable log storage
AWS Transit Gateway	Shared network connectivity
Amazon Route 53	DNS management

Account Vending Flow



Important Guardrails (SCP Examples)

- Deny creating IAM users (force Identity Center).
- Deny disabling CloudTrail.
- Deny deleting log buckets.
- Restrict regions allowed.
- Deny leaving the organization.

Centralized Logging

- Enable organization-wide CloudTrail.
- Store logs in dedicated log archive account (S3).
- Enable AWS Config for all accounts.
- Use S3 Object Lock for immutability.

Contain a Compromised Account



- Identify the blast radius (CloudTrail, GuardDuty, Security Hub).
- Apply restrictive SCP to the affected account.
- Revoke or disable compromised credentials (IAM, STS).
- Isolate network access (NACL, SG, TGW route).
- Preserve evidence (logs, snapshots).
- Rebuild the account if needed using account vending.

Best Practices



- Use multiple accounts with clear purpose.
- Enforce least privilege with SCPs and IAM.
- Centralize security, logging, and monitoring.
- Automate account vending and baseline configuration.
- Regularly review guardrails and IAM access.
- Test account isolation procedures.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



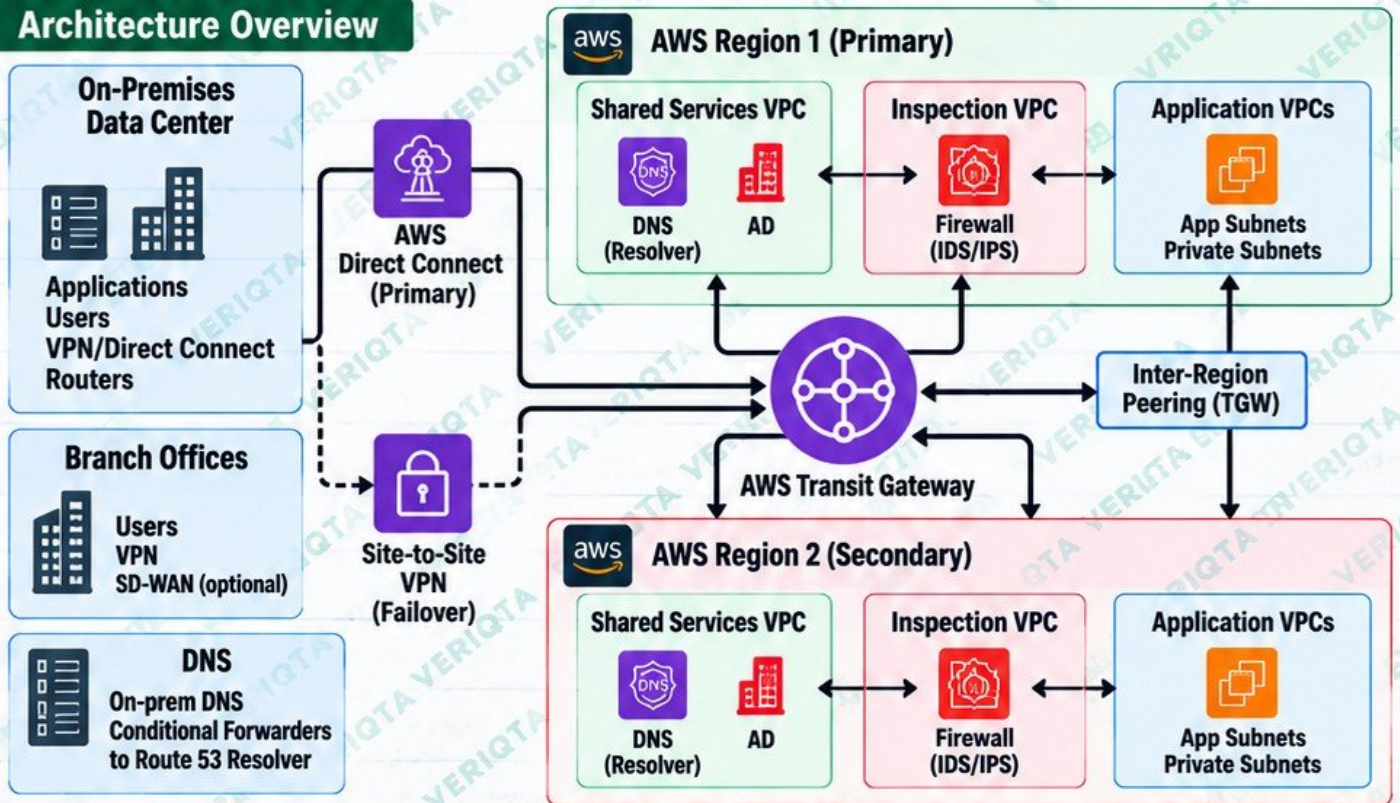
@veriqta_academy

Objective

Design Transit Gateway routing, Direct Connect, VPN failover, DNS resolution, inspection VPCs, and segmented network domains.

Senior Challenge: Diagnose asymmetric routing and prevent inspection bypass during failover.

Architecture Overview



Route Flow (Example)

- 1 On-prem traffic to AWS enters via Direct Connect (primary).
- 2 Traffic is routed to Transit Gateway (TGW).
- 3 TGW routes traffic to Inspection VPC for security checks.
- 4 After inspection, traffic goes to the target Application VPC.
- 5 Return traffic follows the same inspection path.
- 6 If Direct Connect fails, VPN takes over with the same routing and inspection path.

Key AWS Services and Purpose

Service	Purpose
AWS Transit Gateway	Central routing for multi-VPC and on-prem
AWS Direct Connect	Dedicated private connectivity
AWS Site-to-Site VPN	Encrypted backup connection
Amazon Route 53 Resolver	DNS for hybrid environments
Inspection VPC (Firewall)	Traffic inspection (IDS/IPS, NGFW)
VPC Segmentation	Isolate workloads (prod, dev, shared)
AWS Network Firewall	Threat filtering and egress control
AWS Cloud WAN (optional)	Global network management at scale

Common Failure Scenarios

- Asymmetric routing (return traffic bypasses inspection).
- Inspection bypass during failover.
- Incorrect TGW route propagation or route priority.
- DNS resolution using public instead of private endpoints.
- Overlapping CIDR ranges.
- VPN tunnel flapping causing intermittent connectivity.
- Blackhole routes due to missing propagation.

Best Practices

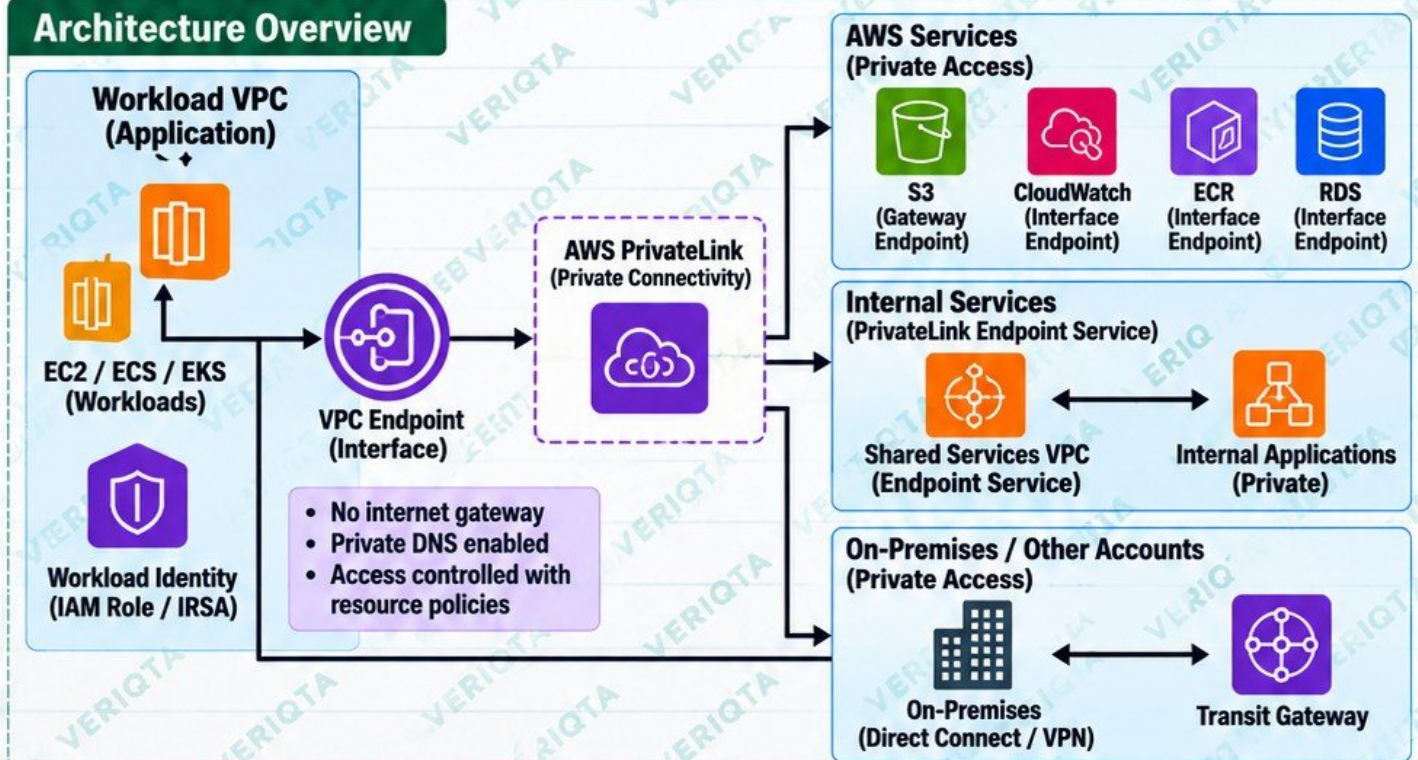
- Use dedicated inspection VPC for ingress and egress.
- Enforce symmetric routing with TGW route tables.
- Use consistent route propagation across attachments.
- Use Route 53 Resolver and conditional forwarding.
- Implement network segmentation (prod, dev, shared).
- Test failover regularly (Direct Connect → VPN).
- Monitor with VPC Flow Logs and AWS Network Firewall logs.
- Use least privilege security groups and NACLs.

Objective

Use PrivateLink, VPC endpoints, workload identities, resource policies, and private DNS to isolate critical services.

Senior Challenge: Eliminate unnecessary internet egress while preserving controlled service access.

Architecture Overview



Key Components

Component	Purpose
 AWS PrivateLink	Private connectivity to services without internet
 VPC Endpoints	Access AWS services privately (Interface and Gateway)
 Resource Policies	Control which principals can access
 Private DNS	Resolve service endpoints to private IPs
 Workload Identities	Use IAM roles (EC2/ECS/EKS/IRSA) instead of long-lived credentials
 Transit Gateway	Private connectivity to on-premises and other VPCs
 Network Policies (SG/NACL)	Restrict traffic to required endpoints

Sample Resource Policy (S3 with VPC Endpoint)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": "arn:aws:s3::my-private-bucket/*",
      "Condition": {
        "StringEquals": {
          "aws:SourceVpce": "vpce-0abc123def456"
        }
      }
    }
  ]
}
```

Internet Egress Elimination Strategy



1. Remove internet gateways from workload subnets.
2. Use VPC endpoints for required AWS services.
3. Use PrivateLink for internal and third-party services.
4. Enforce route tables to prevent 0.0.0.0/0 to internet.
5. Use firewall/inspection only where required.
6. Monitor with VPC Flow Logs for any unexpected egress.

Best Practices



- Use PrivateLink for internal and third-party services.
- Enable private DNS for seamless name resolution.
- Use least privilege resource policies.
- Standardize endpoint deployment with IaC.
- Enforce network segmentation per environment.
- Monitor VPC Flow Logs and DNS logs.
- Regularly audit for unintended internet access.
- Use Interface endpoints for higher security control.



@veriqa_academy



@veriqa_academy



@veriqa_academy



@veriqa_academy



@veriqa_academy



@veriqa_academy

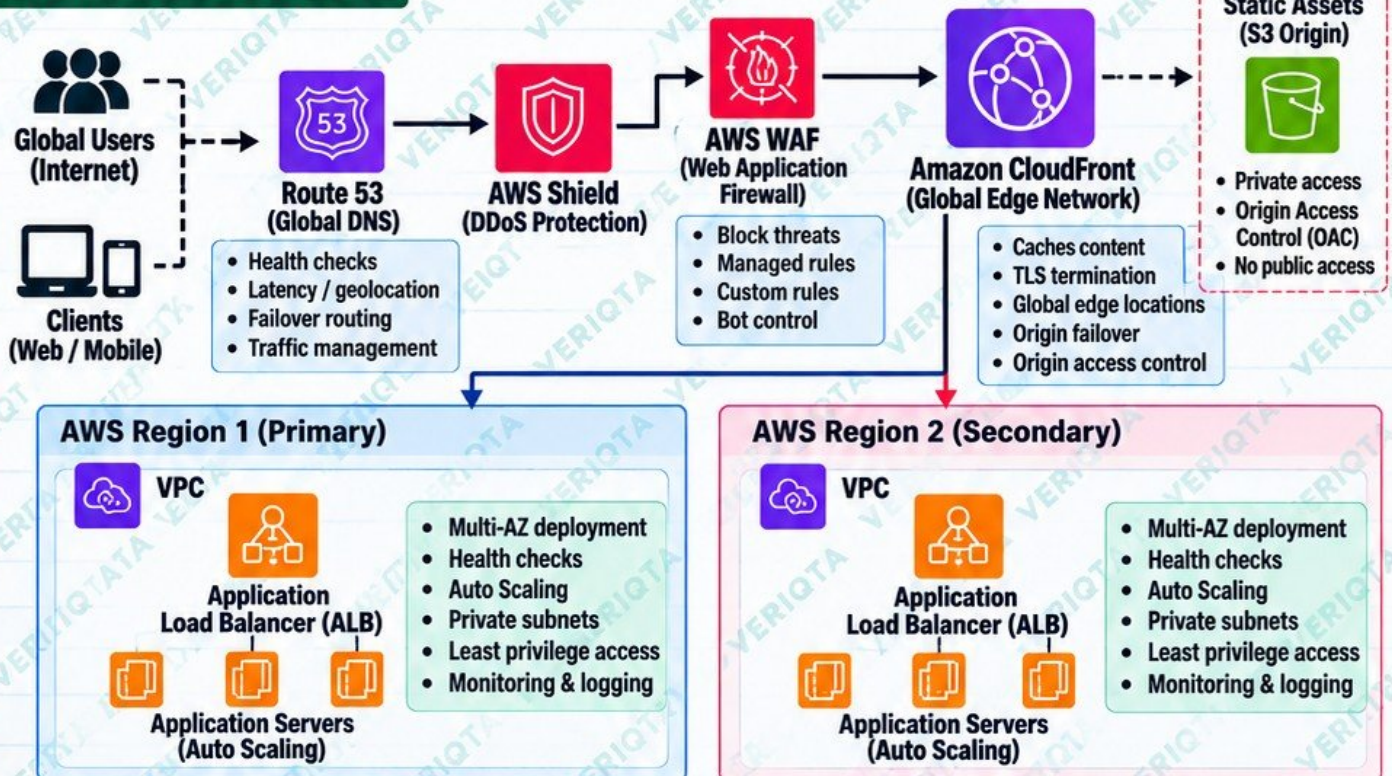
Objective

Design CloudFront, AWS WAF, Shield, Route 53, regional ALBs, and origin protection.



Senior Challenge: Mitigate a regional edge-to-origin failure without overwhelming surviving origins.

Architecture Overview



Key AWS Services

Service	Purpose
Amazon Route 53	Global DNS, health checks, failover, latency routing
AWS Shield	DDoS protection (L3/L4)
AWS WAF	Application layer protection (L7)
Amazon CloudFront	Global CDN, caching, edge locations
Application Load Balancer	Distribute traffic to application servers
Amazon S3 (Origin)	Static content origin with OAC
AWS Global Accelerator (optional)	Improves performance for dynamic traffic
Amazon CloudWatch	Monitoring, metrics, alarms
AWS Certificate Manager	TLS certificates for CloudFront and ALB

Request Flow



Origin Protection Options

S3 Origin	- Use Origin Access Control (OAC) - Block public access - Use bucket policy to allow only CloudFront
ALB Origin	- Restrict origin to CloudFront IP ranges - Use security groups or AWS WAF - Use HTTPS and valid certificates - Use custom headers if needed

Common Failure Scenarios



1. Regional origin failure (ALB/EC2/AZ outage).
2. Health checks misconfigured (false healthy).
3. Traffic overload on surviving region.
4. WAF rules blocking legitimate traffic.
5. Origin latency causing high error rates.
6. DNS misconfiguration or slow propagation.

Best Practices



1. Deploy origins in multiple regions with independent stacks.
2. Use Route 53 health checks with appropriate thresholds.
3. Configure CloudFront origin failover and custom error handling.
4. Implement auto scaling with capacity headroom.
5. Use AWS WAF and Shield for layered protection.
6. Monitor edge, origin, and end-to-end latency and error rates.
7. Run regular failover and load testing.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



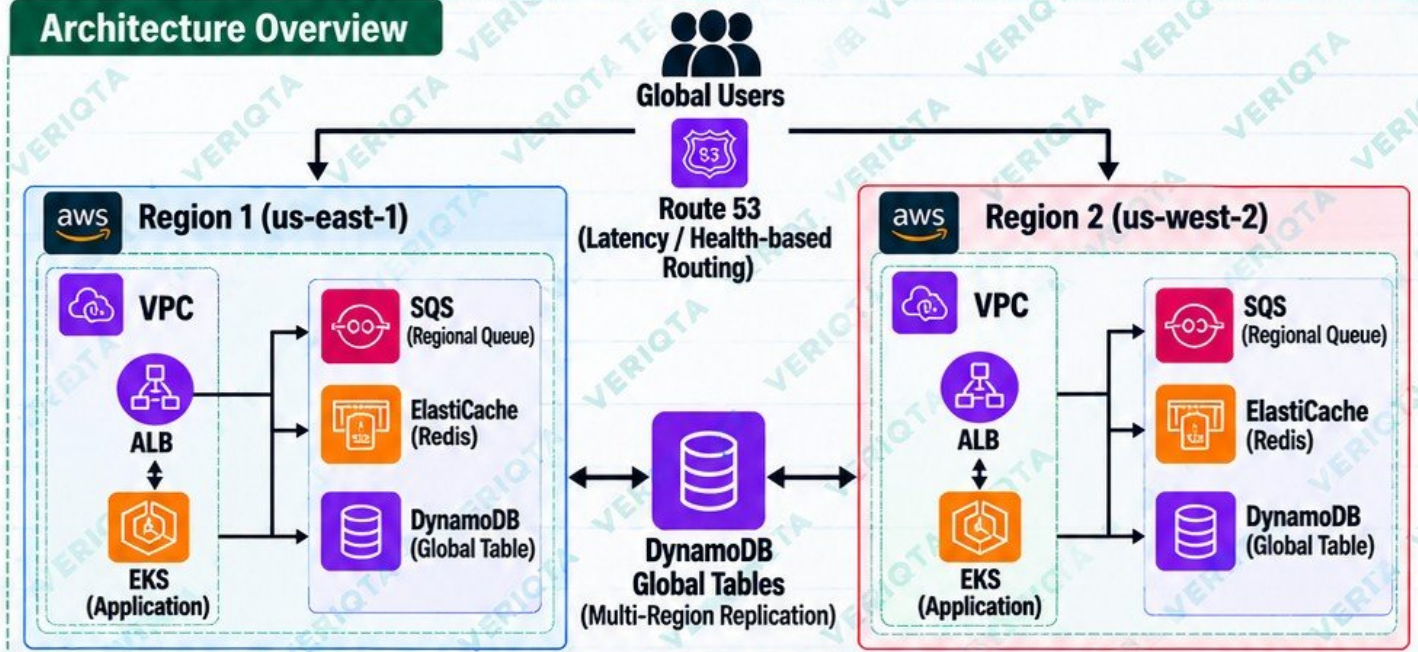
@veriqta_academy

Objective

Deploy independently operating regional application stacks with DynamoDB Global Tables, regional queues, and traffic steering.

Senior Challenge: Resolve concurrent-write conflicts, replication lag, and regional dependency failures.

Architecture Overview



Key AWS Services

Service	Purpose
Amazon Route 53	Global DNS and traffic steering
DynamoDB Global Tables	Multi-region data replication
Amazon EKS	Regional application workloads
Application Load Balancer	Distribute traffic to services
Amazon SQS	Regional message queues
Amazon ElastiCache	Caching and session storage
Amazon CloudWatch	Monitoring, metrics, alarms
AWS Global Accelerator	Improved global performance (optional)

Data Flow (Write Operation)



Traffic Steering Options

Latency-based	Send users to the lowest latency region.
Health-based	Fail over when a region is unhealthy.
Weighted routing	Shift a percentage of traffic between regions.
Geolocation	Route based on user location.
AWS Global Accelerator	Static anycast IP with built-in health checks.

Common Failure Scenarios



1. Region outage (compute or network).
2. DynamoDB replication lag.
3. Concurrent writes causing data conflicts.
4. Queue backlog in one region.
5. DNS misrouting or slow propagation.
6. Dependency service failure (RDS, external API).

Design Considerations



- Use idempotent writes to handle retries.
- Design for eventual consistency.
- Use conflict resolution (last-writer-wins or custom logic).
- Isolate regional dependencies.
- Monitor replication lag and set alarms.
- Regularly test regional failover.
- Validate failover and fallback procedures.

Conflict Resolution Strategies

Strategy	When to Use
Last-writer-wins	Simple use cases, low conflict rate.
Application-based merge	When data requires custom logic.
Versioning with timestamps	Track and resolve concurrent updates.
Idempotent writes	Prevent duplicate or conflicting changes.

Best Practices



- Keep each region fully functional and independent.
- Use DynamoDB Global Tables for multi-region data.
- Implement retries with backoff and idempotency.
- Monitor replication lag and set alarms.
- Regularly test regional failover.
- Avoid cross-region synchronous dependencies.
- Define clear RTO and RPO for each service.



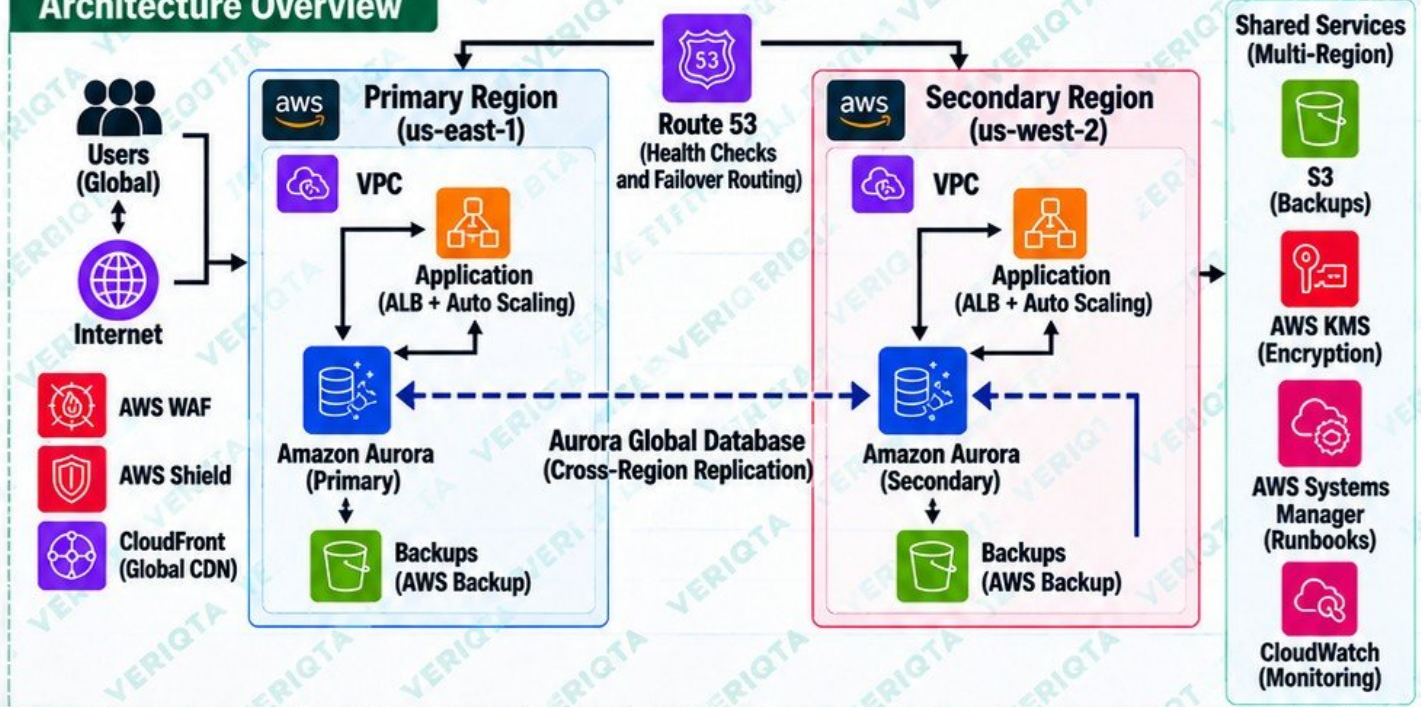
Objective

Implement Aurora Global Database, infrastructure recovery, backup validation, failover runbooks, and recovery testing.



Senior Challenge: Demonstrate achievable RPO and RTO, including application recovery and data reconciliation.

Architecture Overview



Key AWS Services

Service	Purpose
Amazon Aurora Global Database	Cross-region database replication
Amazon Route 53	Health checks and failover routing
AWS Backup	Centralized backups and policies
Amazon S3	Long-term backup storage
AWS KMS	Encryption for data and backups
AWS CloudWatch	Monitoring, alarms, and dashboards
AWS Systems Manager	Automation and runbooks
AWS Elastic Load Balancer (ALB)	Distribute traffic to application servers
Auto Scaling	Maintain application capacity
AWS CloudFront + WAF + Shield	Global edge, security, and DDoS protection

DR Flow (Normal and Failover)

Normal Operation (Primary)

1. Users access via Route 53.
2. Traffic goes to primary region.
3. Application writes to Aurora (primary).
4. Data replicates to secondary region (Aurora Global Database).
5. Backups run automatically.

Failover (Secondary Takes Over)

1. Route 53 detects failure (health checks).
2. Traffic is redirected to secondary region.
3. Promote Aurora secondary to primary (if needed).
4. Application uses secondary infrastructure.
5. Continue operations and restore in primary region later.

Recovery Targets



RPO
(Recovery Point Objective)
Target: 0 – 5 minutes
(Using Aurora Global Database)

RTO
(Recovery Time Objective)
Target: 15 – 60 minutes
(Including application recovery)

Common Failure Scenarios



1. Regional outage (compute, network, or database).
2. Aurora replication lag.
3. Corrupted data or accidental deletion.
4. DNS or routing misconfiguration.
5. Incomplete backups or failed restore.
6. Application dependency failures (cache, queue, external API).

Backup Validation Steps



1. Verify automated backups are completing.
2. Test restore to a separate environment.
3. Validate application functionality after restore.
4. Regularly test cross-region failover.
5. Document and update runbooks.

Best Practices



- Use Aurora Global Database for low RPO.
- Automate recovery with runbooks (SSM).
- Test failover regularly (game days).
- Monitor replication lag and set alerts.
- Keep infrastructure as code for fast rebuilds.
- Validate backup restores and application health.
- Use Route 53 health checks with proper thresholds.
- Document RTO/RPO and communicate with stakeholders.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy

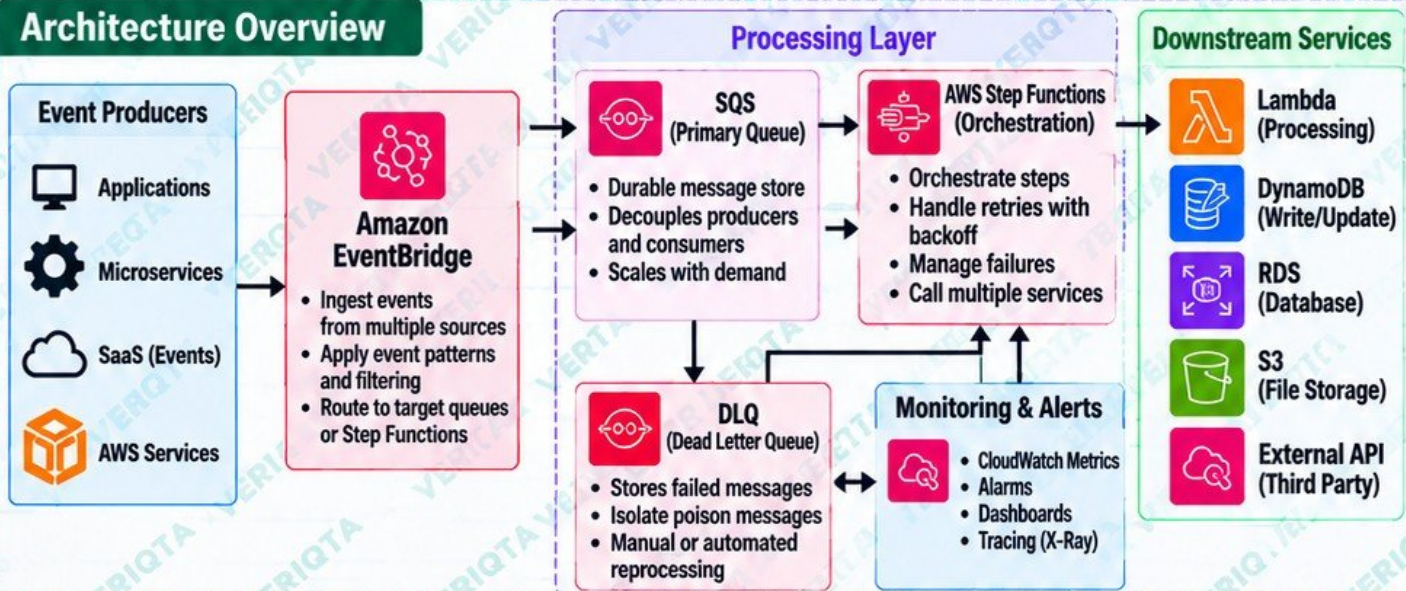
Objective

Combine EventBridge, SQS, Step Functions, DLQs, idempotency controls, and bounded retries.



Senior Challenge: Recover from duplicate deliveries, poison messages, and downstream saturation without losing business transactions.

Architecture Overview



Key AWS Services

Service	Purpose
Amazon EventBridge	Event ingestion, filtering and routing
Amazon SQS	Reliable message queue
AWS Step Functions	Workflow orchestration and retries
Dead Letter Queue (DLQ)	Store failed/poison messages
AWS Lambda	Event processing and business logic
Amazon DynamoDB	Idempotency store and application data
Amazon S3	File storage and audit logs
Amazon CloudWatch	Monitoring, metrics and alarms
AWS X-Ray	Distributed tracing
AWS IAM	Secure access and least privilege

Event Processing Flow

- Producers publish events to EventBridge.
- EventBridge filters and routes to SQS.
- Messages are processed by Step Functions.
- Step Functions calls downstream services (Lambda, DynamoDB, APIs, etc).
- Successful processing updates the idempotency record.
- Failed messages are retried with exponential backoff (bounded retries).
- Messages that still fail go to DLQ.
- CloudWatch monitors the entire flow with alarms and tracing.

Idempotency Strategy



- Use a unique event ID (e.g., orderId, transactionId).
- Store processed event IDs in DynamoDB with TTL.
- Check idempotency record before processing.
- Use conditional writes to prevent duplicates.
- Design downstream operations to be idempotent.

DLQ Handling



- Send failed messages to a dedicated DLQ.
- Analyze error patterns and root cause.
- Fix the issue and reprocess messages safely.
- Use a replay mechanism (Step Functions or Lambda).
- Set alarms for DLQ depth and age.

Common Failure Scenarios



- Duplicate event deliveries (at-least-once delivery).
- Poison messages (always fail).
- Downstream service throttling or saturation.
- Partial failures in multi-step workflows.
- Event ordering issues.
- Message backlog and processing delays.

Best Practices



- Design idempotent consumers.
- Use bounded retries with exponential backoff.
- Isolate and monitor DLQs.
- Implement circuit breakers for downstream services.
- Use reserved concurrency and throttling controls.
- Monitor key metrics (queue depth, age, failure rate, latency).
- Test failure scenarios regularly (chaos / game days).



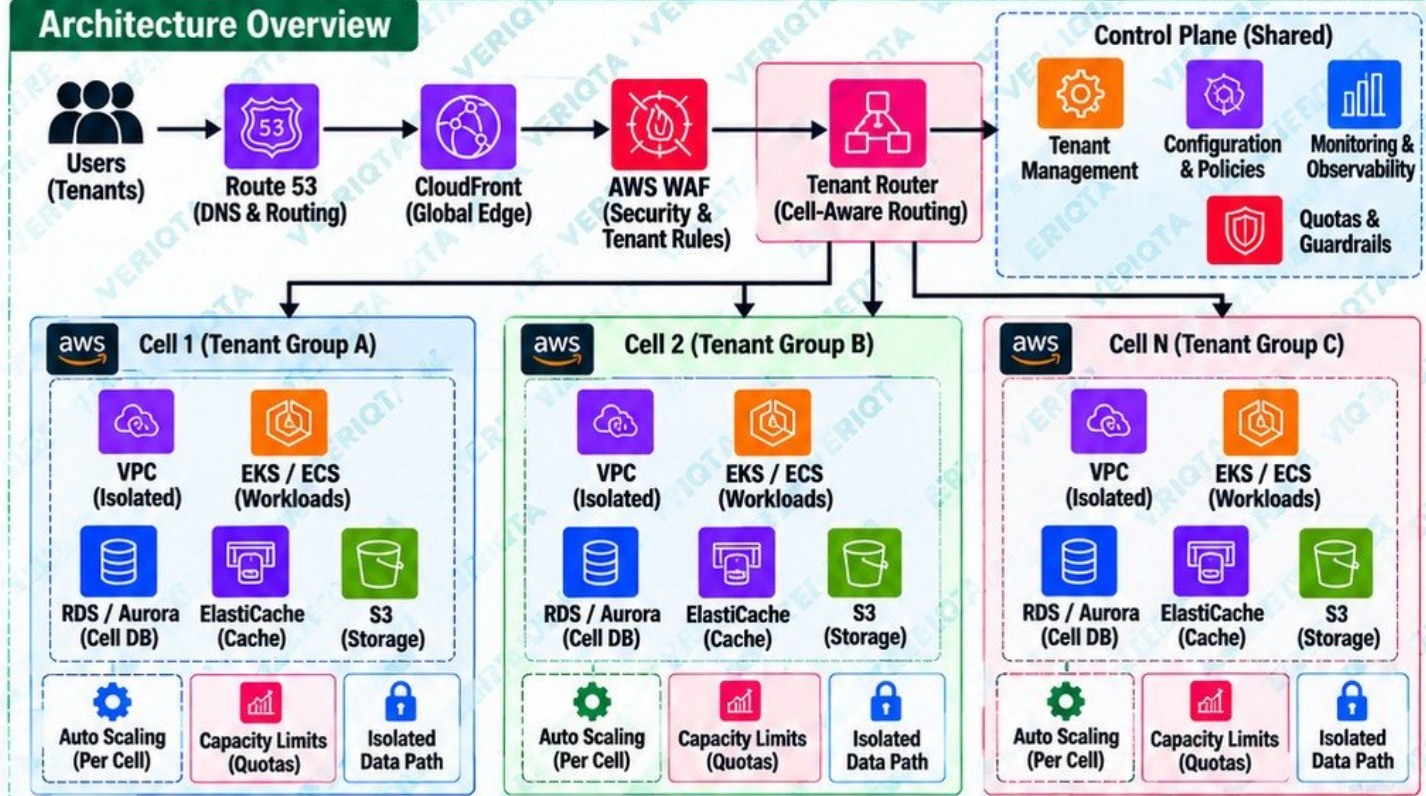
Objective

Partition workloads into independent cells with tenant routing, per-cell capacity, quotas, and isolated data paths.



Senior Challenge: Prevent a single tenant or deployment defect from cascading across the platform.

Architecture Overview



Key AWS Services

Service	Purpose
Route 53	DNS and tenant/cell routing
CloudFront	Global edge and caching
AWS WAF	Security and tenant-based rules
VPC	Isolated network per cell
EKS / ECS	Cell workloads (containerized)
RDS / Aurora	Isolated database per cell
ElastiCache	Caching per cell
S3	Storage per cell
CloudWatch	Monitoring and alarms per cell
AWS IAM	Tenant isolation and least privilege
Service Quotas	Per-cell resource limits
AWS Organizations	Account/region governance (optional)

Request Flow (Tenant to Cell)

- 1 User request arrives at Route 53.
- 2 CloudFront caches and forwards the request.
- 3 AWS WAF applies security and tenant rules.
- 4 Tenant router directs traffic to the correct cell.
- 5 Request is processed by cell workloads (EKS/ECS).
- 6 Cell uses its own database, cache, and storage.
- 7 Monitoring tracks per-cell metrics and limits.
- 8 If a cell is unhealthy, traffic is routed to a healthy cell or the tenant is isolated based on policy.

Isolation Controls

- Dedicated VPC per cell
- Separate compute, database, and storage
- Network isolation (no cross-cell access)
- Resource quotas per cell
- IAM boundaries and scoped roles
- Rate limiting and throttling
- Independent monitoring and alerting
- Automated isolation on failure

Common Failure Scenarios



1. Tenant causes high traffic or resource exhaustion.
2. Faulty deployment or bad release in one cell.
3. Database performance issues or connection leaks.
4. Noisy neighbor consuming shared resources.
5. Cascading failures through shared services.
6. Misconfigured quotas or insufficient limits.

Best Practices



1. Keep cells independent (compute, data, network).
2. Set and enforce per-cell quotas and limits.
3. Use automated routing and health checks.
4. Implement fast isolation for unhealthy cells.
5. Monitor per-cell SLIs, SLOs, and error budgets.
6. Test failure scenarios regularly (chaos testing).
7. Design clear runbooks for cell isolation and recovery.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy

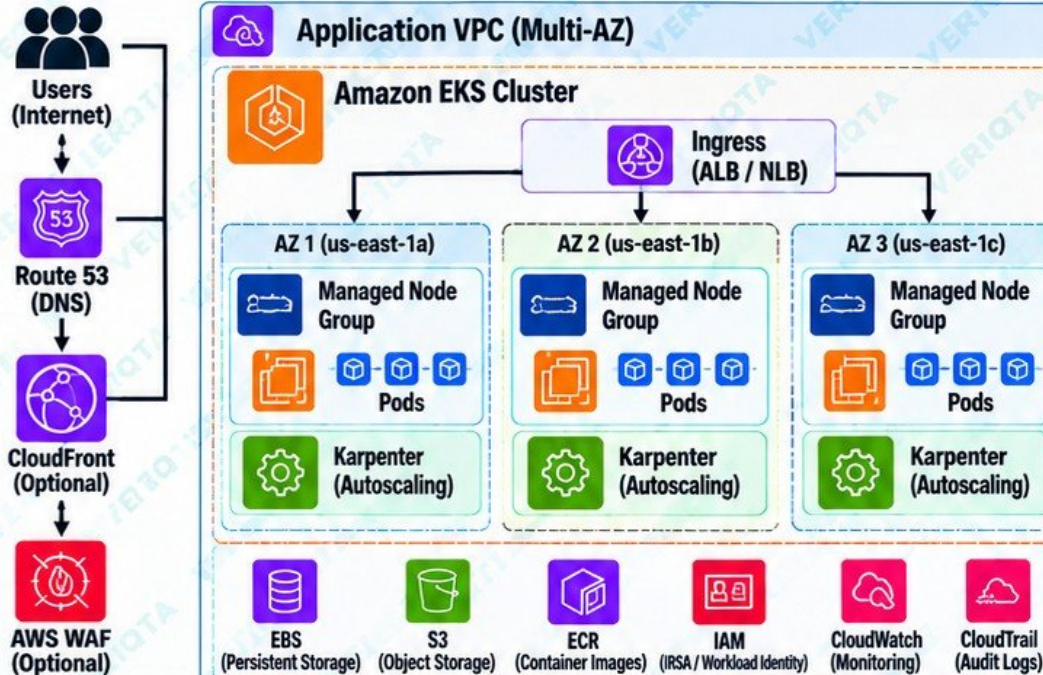
Objective

Design managed node groups, Karpenter, topology spread, PDBs, ingress, workload identity, and node lifecycle controls.



Senior Challenge: Maintain service availability through node loss, AZ impairment, and cluster upgrades.

Architecture Overview



Supporting Services

- Karpenter (Node Provisioning)
- Cluster Autoscaler (Optional)
- AWS Load Balancer Controller
- Pod Disruption Budgets (PDBs)
- IAM Roles for Service Accounts (IRSA)
- AWS Systems Manager (Node Lifecycle)

Key AWS Services

Service	Purpose
Amazon EKS	Managed Kubernetes control plane
Managed Node Groups	Reliable worker nodes per AZ
Karpenter	Automatic, efficient node provisioning
ALB / NLB	Ingress traffic to services
Route 53	DNS and traffic routing
Amazon VPC	Isolated network environment
Amazon EBS	Persistent storage for workloads
Amazon ECR	Container image registry
IAM (IRSA)	Workload identity for pods
Amazon CloudWatch	Monitoring, metrics and alarms
Amazon CloudTrail	Audit and API activity
AWS Systems Manager	Node lifecycle and patching

EKS Components

- Managed Node Groups**
 - Multi-AZ deployment
 - Auto repair and upgrades
 - On-demand / Spot instances
- Karpenter**
 - Dynamic node provisioning
 - Optimizes cost and capacity
 - Handles scaling and consolidation
- Topology Spread**
 - Distribute pods across AZs/nodes
 - Improve resilience and availability
- Pod Disruption Budgets (PDBs)**
 - Maintain minimum pod availability
 - Prevent voluntary disruptions
- Ingress (ALB/NLB)**
 - Expose services with high availability
 - Integrate with AWS Load Balancer Controller
- Workload Identity (IRSA)**
 - Secure access to AWS services
 - No hardcoded credentials
- Node Lifecycle Controls**
 - Rolling upgrades
 - Draining and cordonning
 - Automated patching with SSM

Deployment Flow

1. Set up VPC with private subnets in multiple AZs.
2. Create EKS cluster with OIDC and IRSA.
3. Deploy managed node groups and Karpenter.
4. Configure storage, networking and ingress.
5. Implement topology spread and PDBs.
6. Deploy workloads with workload identity.
7. Set up monitoring, logging and alerts.
8. Test node loss, AZ failure and cluster upgrade scenarios.

Common Failure Scenarios



1. Single AZ node loss causing pod unavailability.
2. Cluster upgrade draining too many pods.
3. Insufficient node capacity or slow scaling.
4. Misconfigured PDBs blocking deployment.
5. Ingress/LB failure or misconfiguration.
6. IAM/IRSA misconfiguration causing access issues.

Best Practices



1. Run workloads across multiple AZs.
2. Use Karpenter with Spot and On-Demand.
3. Configure topology spread and PDBs correctly.
4. Implement proper resource requests and limits.
5. Automate node lifecycle and patching.
6. Monitor cluster health, node status and pod disruptions.
7. Regularly test failure scenarios.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy

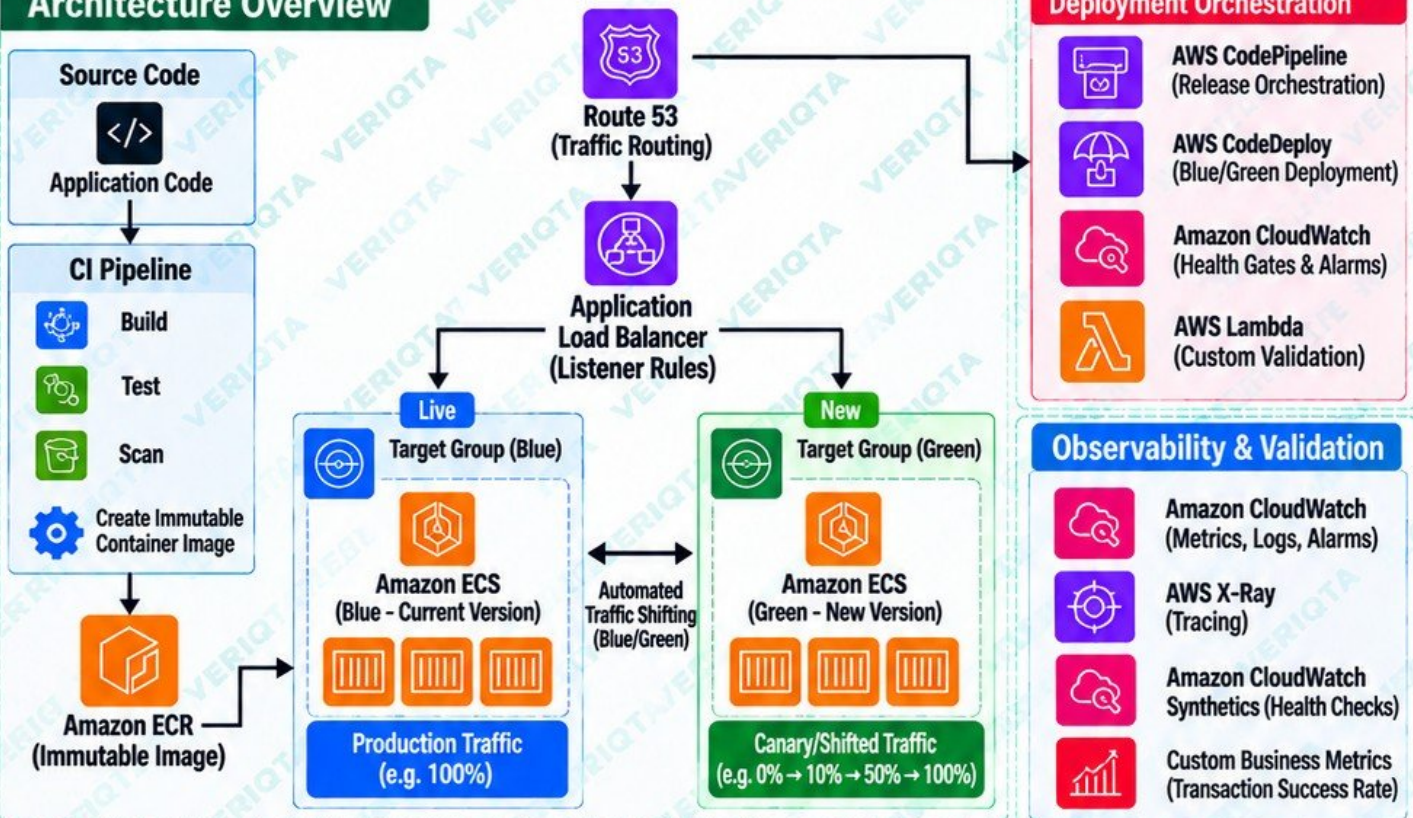
Objective

Implement immutable artifacts, ECS blue-green releases, traffic shifting, automated health gates, and rollback controls.



Senior Challenge: Detect a release that passes infrastructure health checks but causes business transaction failures.

Architecture Overview



Key AWS Services

Service	Purpose
 Amazon ECS	Run containerized applications
 AWS CodePipeline	CI/CD orchestration
 AWS CodeDeploy	Blue-green deployments
 Amazon ECR	Immutable container images
 Application Load Balancer	Traffic routing and health checks
 Route 53	DNS and traffic management
 Amazon CloudWatch	Monitoring and health gates
 AWS Lambda	Custom validation logic
 AWS X-Ray	Distributed tracing
 AWS IAM	Access control and permissions

Deployment Flow

- Developer pushes code to repository.
- CI pipeline builds, tests, scans and creates an immutable image.
- Image is stored in Amazon ECR.
- CodeDeploy deploys new version to green target group.
- Automated health checks validate infrastructure and application health.
- Traffic is gradually shifted from blue to green (e.g. 0% → 10% → 50% → 100%).
- Business metrics are monitored during the shift.
- If issues are detected, automatically rollback to blue.

Traffic Shifting Strategies

 Linear	Increase traffic by a fixed percentage over time.
 Canary	Send a small percentage of traffic first.
 All-at-once	Shift 100% of traffic after validation.
 Weighted DNS	Use Route 53 weighted routing for multi-region.
 Header-based	Route specific users or cohorts.
 Feature flag	Enable features for specific user groups.

Common Failure Scenarios



- Release passes health checks but breaks business transactions.
- Increased error rate after traffic shift.
- Latency or performance degradation.
- Incorrect target group or listener configuration.
- Dependency failure (database, external API).
- Rollback takes too long or is incomplete.

Best Practices



- Use immutable container images.
- Automate health gates with both technical and business metrics.
- Implement gradual traffic shifting.
- Monitor real user transactions and SLIs/SLOs.
- Set automatic rollback on failure thresholds.
- Test rollback regularly.
- Keep deployments idempotent and repeatable.



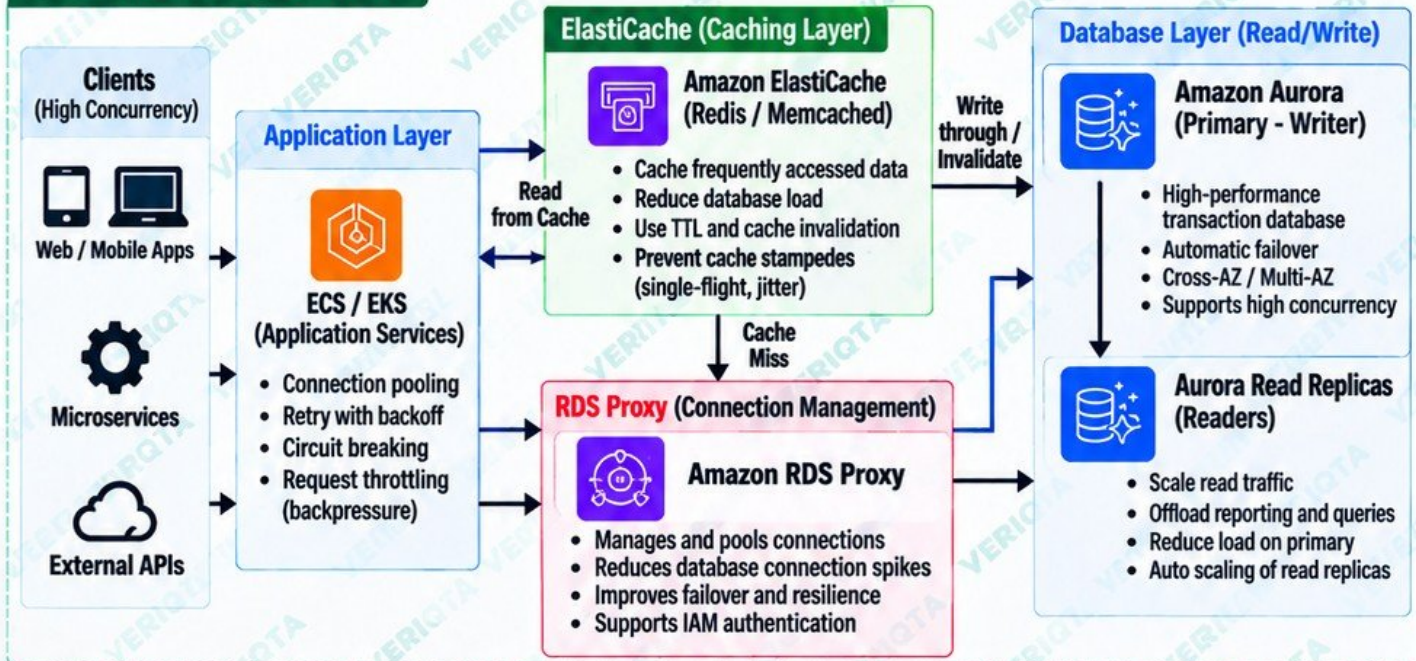
Objective

Combine Aurora, RDS Proxy, ElastiCache, read scaling, connection budgeting, and backpressure.



Senior Challenge: Investigate p99 latency spikes caused by lock contention, pool exhaustion, and cache stampedes.

Architecture Overview



Key AWS Services

Service	Purpose
 Amazon Aurora	Primary transactional database
 Amazon RDS	Managed relational database
 RDS Proxy	Connection pooling and management
 Amazon ElastiCache	Caching layer (Redis/Memcached)
 ECS / EKS	Application runtime
 Application Load Balancer	Distribute traffic to applications
 Amazon CloudWatch	Monitoring and alarms
 AWS Systems Manager	Operational automation
 AWS Secrets Manager	Manage database credentials

High-Concurrency Data Flow

- 1 Users send requests to the application.
- 2 Application checks ElastiCache for data.
- 3 If cache miss, application queries Aurora through RDS Proxy.
- 4 Reads are scaled using Aurora read replicas.
- 5 Connections are pooled and managed by RDS Proxy.
- 6 Write operations go to Aurora primary.
- 7 Cache is updated or invalidated after writes.
- 8 Backpressure and throttling protect the database during spikes.

Connection & Scaling Strategy

-  Use RDS Proxy to pool and reuse connections.
-  Set connection limits per service.
-  Use read replicas for read-heavy workloads.
-  Implement application-level connection pooling.
-  Use auto scaling for application tier.
-  Enforce rate limiting and backpressure.
-  Monitor active connections and queue depth.
-  Plan capacity for peak concurrency.

Common Failure Scenarios



1. p99 latency spikes due to lock contention.
2. Database connection pool exhaustion.
3. Cache stampedes after cache expiration.
4. Long-running transactions blocking resources.
5. Read replica lag causing stale reads.
6. Sudden traffic surge overwhelming the database.

Investigation Checklist



1. Check p99 latency and error rates.
2. Review database locks and long queries.
3. Analyze RDS Proxy connection metrics.
4. Check ElastiCache hit ratio and evictions.
5. Look for cache stampedes in logs.
6. Verify read replica lag.
7. Review application logs and throttling events.
8. Correlate with traffic patterns and deployments.

Best Practices



- Use caching for hot data.
- Set proper TTL and cache invalidation.
- Implement connection pooling.
- Use RDS Proxy and IAM authentication.
- Scale reads with Aurora read replicas.
- Set and enforce connection limits.
- Implement backpressure and rate limiting.
- Monitor p99 latency, DB locks, and cache hit ratio.
- Test under high-concurrency load.
- Prepare runbooks for common scenarios.



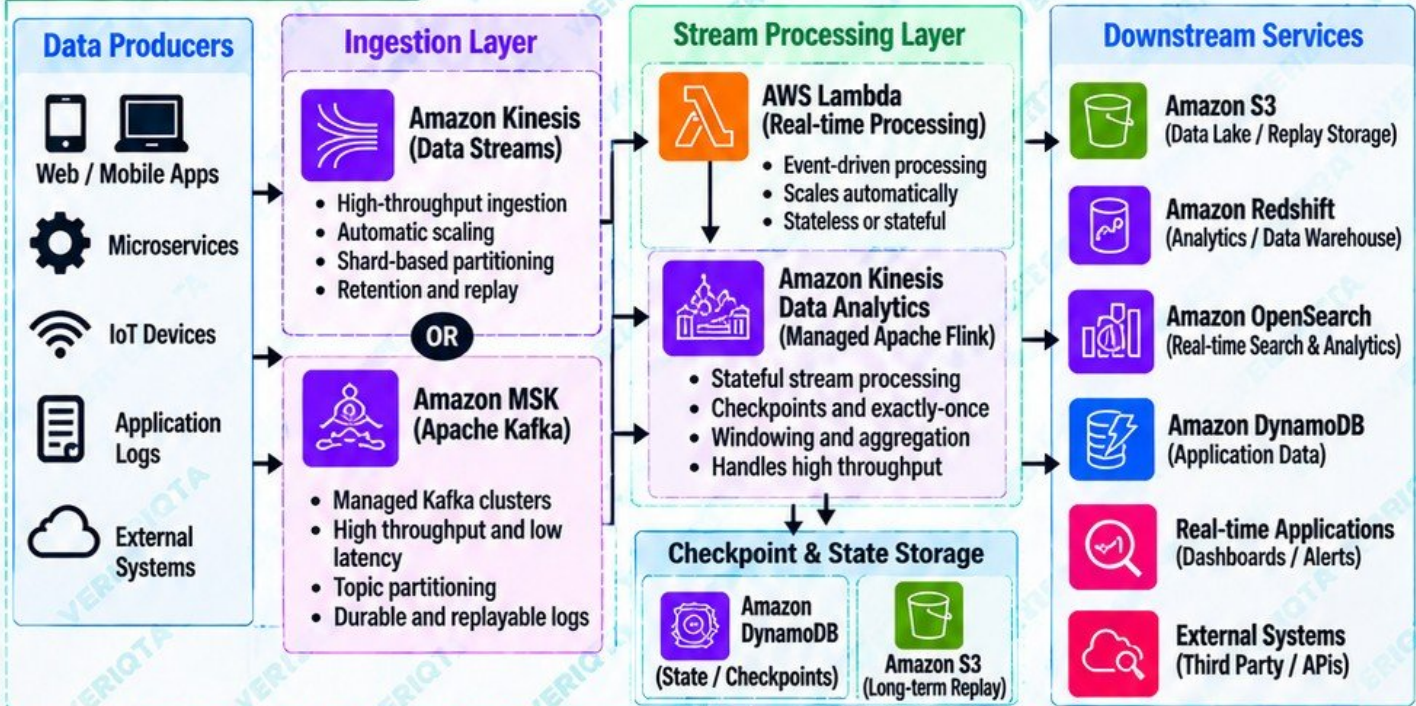
Objective

Design ingestion with Kinesis or MSK, stream processing, checkpoints, partition strategies, and replayable storage.



Senior Challenge: Handle consumer lag, skewed partitions, processing failures, and replay without corrupting downstream results.

Architecture Overview



Key AWS Services

Service	Purpose
 Amazon Kinesis Data Streams	Real-time data ingestion and streaming
 Amazon MSK (Kafka)	Managed Kafka for high-throughput streaming
 AWS Lambda	Event-driven processing
 Amazon Kinesis Data Analytics	Stream processing (Apache Flink)
 Amazon DynamoDB	State storage and checkpoints
 Amazon S3	Durable replayable storage (data lake)
 Amazon Redshift	Analytics and reporting
 Amazon OpenSearch	Real-time search and analytics
 Amazon CloudWatch	Monitoring, metrics and alarms
 AWS IAM	Access control and permissions

Streaming Data Flow

- Producers send data to Kinesis or MSK.
- Data is partitioned across shards/partitions.
- Stream processing reads and processes records with checkpoints.
- Processed data is written to downstream services (S3, Redshift, DynamoDB, etc.).
- Offsets and state are checkpointed periodically.
- Failed records are retried or sent to a DLQ.
- Data can be replayed from retention for backfills or recovery.
- Downstream systems consume processed data for analytics and applications.

Partition & Scaling Strategy

-  Choose a good partition key (e.g., user ID, tenant ID).
-  Avoid hot partitions and skew.
-  Scale shards/partitions based on throughput and consumer lag.
-  Use consistent hashing when needed.
-  Monitor partition distribution and rebalance.
-  Use enhanced fan-out (Kinesis) for multiple consumers.
-  Plan retention for replay and backfill requirements.

Common Failure Scenarios



- Consumer lag due to high traffic or slow processing.
- Skewed partitions causing uneven load.
- Processing failures and record loss.
- Poison messages blocking consumers.
- Checkpoint failures causing duplicate processing.
- Replay causing data corruption or inconsistent results.

Investigation Checklist



- Check consumer lag and throughput.
- Analyze partition distribution and skew.
- Review processing error logs.
- Verify checkpoint and offset commits.
- Inspect DLQ for failed records.
- Validate data consistency after replay.
- Check downstream system impact.
- Correlate with recent deployments or configuration changes.

Best Practices



- Design a good partition strategy.
- Implement idempotent processing.
- Use checkpoints for exactly-once.
- Set up DLQ for failed records.
- Monitor consumer lag, throughput and errors.
- Test replay and backfill regularly.
- Use proper retention periods.
- Implement alerting and autoscaling.



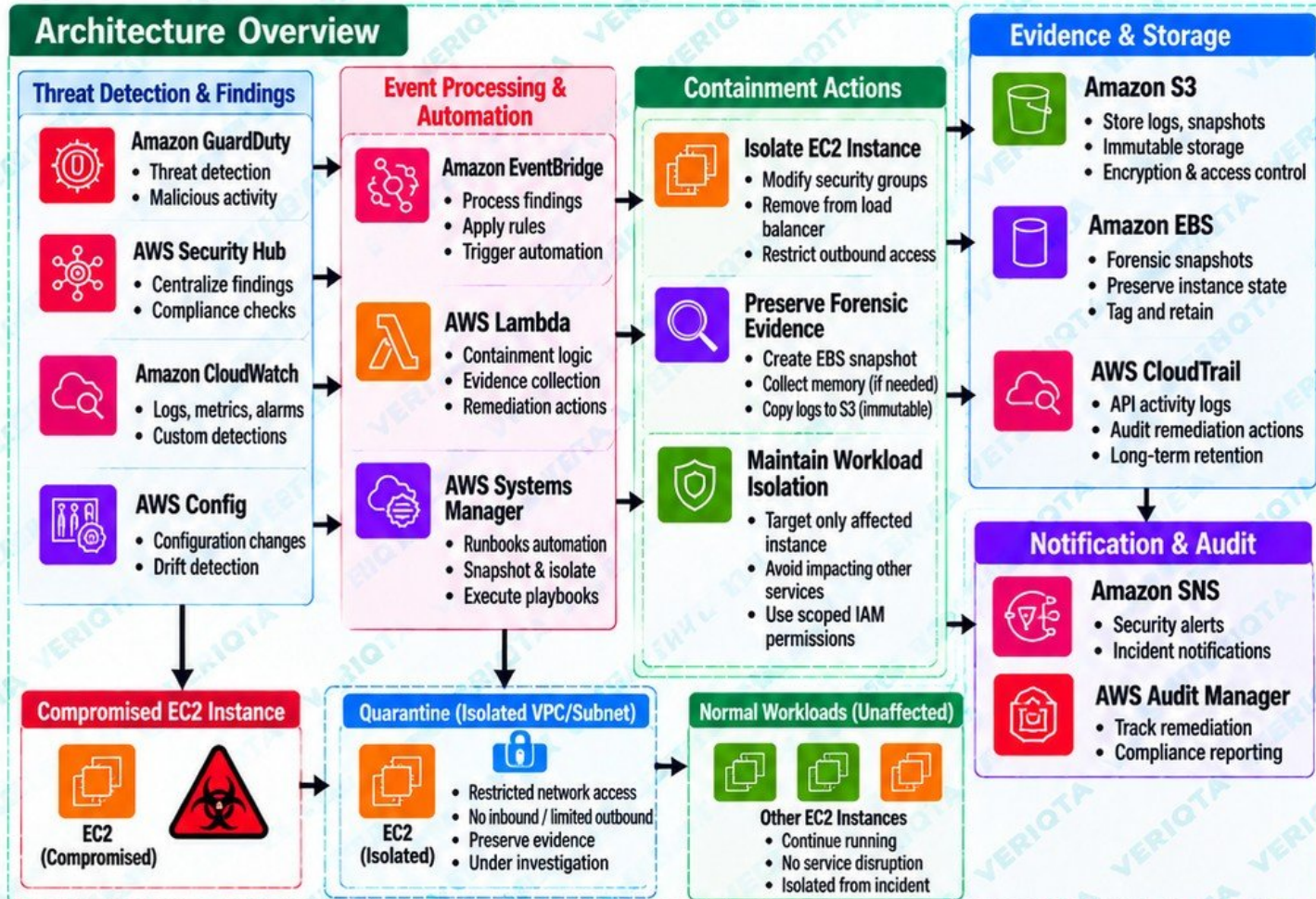
Objective

Integrate GuardDuty, Security Hub, EventBridge, IAM controls, forensic preservation, and audited remediation workflows.



Senior Challenge: Quarantine a compromised EC2 instance without destroying evidence or disrupting unrelated workloads.

Architecture Overview



Key AWS Services

Service	Purpose
 Amazon GuardDuty	Threat detection and findings
 AWS Security Hub	Centralized security findings
 Amazon EventBridge	Event-driven automation
 AWS Lambda	Containment and remediation logic
 AWS Systems Manager	Automated runbooks and commands
 Amazon EC2	Compute instance (target)
 Amazon S3	Forensic evidence storage
 Amazon EBS	Instance snapshots (forensics)
 AWS IAM	Access control and least privilege
 AWS CloudTrail	Audit logs and API activity
 Amazon SNS	Notifications and alerts
 AWS Audit Manager	Compliance and audit tracking

Containment Workflow

- GuardDuty/Security Hub generates a finding.
- EventBridge matches the rule and triggers Lambda.
- Lambda validates the finding and collects instance details.
- Isolate the instance (security groups, remove from LB).
- Create EBS snapshot and collect logs to S3.
- Tag the instance and record evidence.
- Notify the security team via SNS.
- Track all actions in CloudTrail and Audit Manager.

Best Practices

- Use least privilege IAM roles for automation.
- Preserve forensic evidence before making changes.
- Isolate only the affected instance.
- Use immutable storage (S3 with versioning).
- Automate with EventBridge and Lambda.
- Log all actions in CloudTrail.
- Test containment in a non-production environment.
- Ensure incident response runbooks are approved.

Common Failure Scenarios

- Instance terminated before evidence is collected.
- Overly broad security group changes.
- Automation disrupts unrelated workloads.
- Missing or incomplete forensic data.
- Insufficient IAM permissions.
- Lack of audit logs for remediation actions.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy

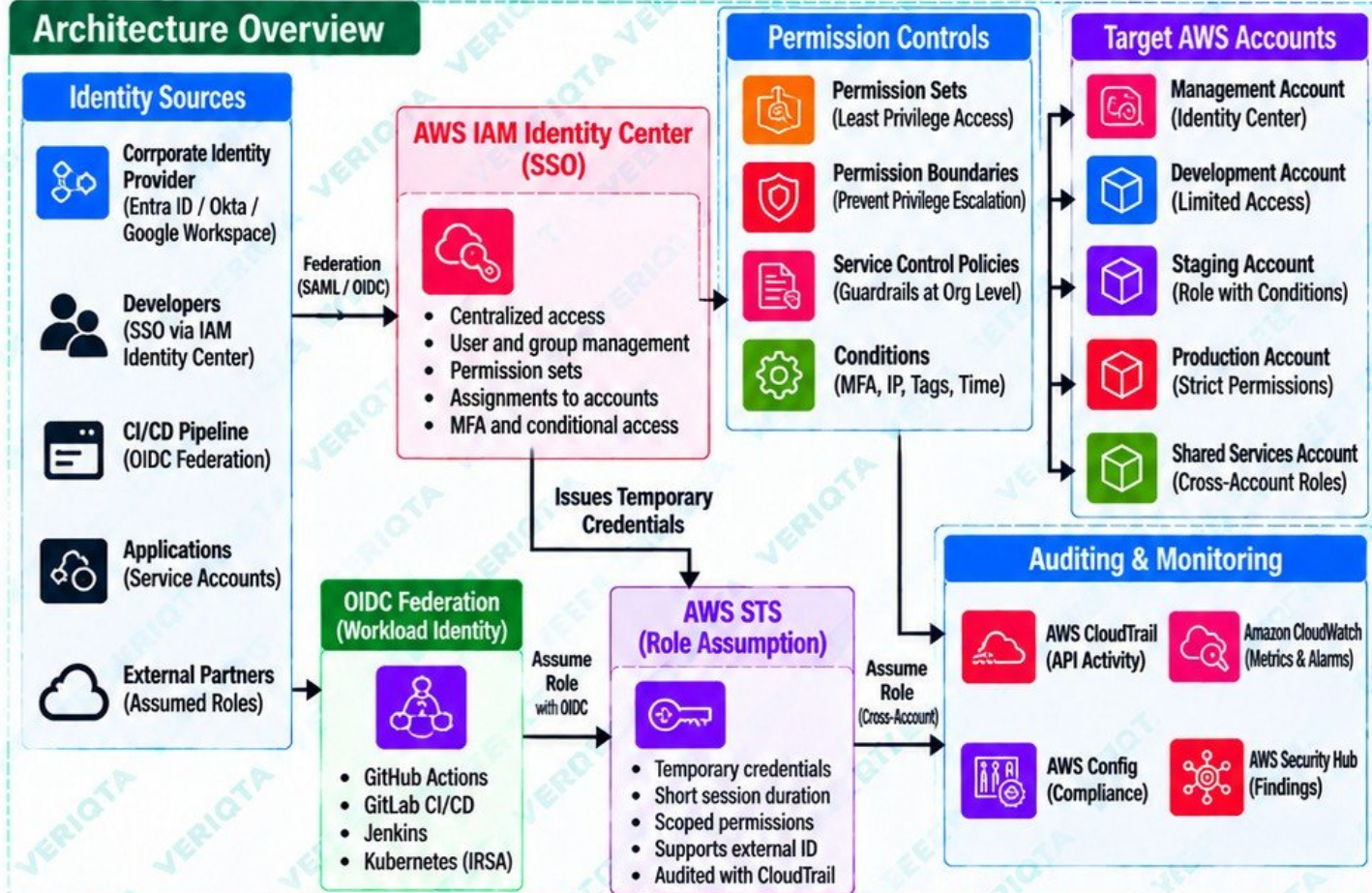
Objective

Design IAM Identity Center, OIDC federation, STS role assumption, permission boundaries, and cross-account trust.



Senior Challenge: Remove long-lived CI/CD credentials and prevent privilege escalation through misconfigured trust policies.

Architecture Overview



Key AWS Services

Service	Purpose
AWS IAM Identity Center	SSO and centralized access
AWS STS	Temporary credential & role assumption
Amazon Cognito	OIDC for application users
IAM Roles	Cross-account and service access
IAM Permission Boundaries	Limit maximum permissions
AWS Organizations (SCPs)	Guardrails across accounts
Amazon EKS (IRSA)	Workload identity for Kubernetes
GitHub Actions (OIDC)	Federation for CI/CD
AWS CloudTrail	Audit and API activity logs
AWS Config	Detect misconfigurations
AWS Security Hub	Security findings and compliance
Amazon CloudWatch	Monitoring and alarms

Access Flow

- 1 User or workload authenticates with identity provider.
- 2 IAM Identity Center or OIDC federation provides access.
- 3 Assume role via AWS STS with temporary credentials.
- 4 Permissions are enforced via permission sets, boundaries and SCPs.
- 5 Access target AWS account and required resources.
- 6 All actions are logged in CloudTrail and monitored.
- 7 Permissions are regularly reviewed and rotated.
- 8 No long-lived credentials are used in CI/CD or applications.

Best Practices



- Use IAM Identity Center for human access.
- Use OIDC federation for CI/CD and workloads.
- Enforce least privilege with permission sets.
- Apply permission boundaries to roles.
- Use SCPs for organization guardrails.
- Enable MFA and conditional access.
- Rotate and limit session duration.
- Monitor with CloudTrail, Config and Security Hub.
- Regularly review trust policies and permissions.
- Remove all long-lived access keys.

Common Failure Scenarios



1. Long-lived CI/CD access keys.
2. Overly permissive trust policies.
3. Missing or weak permission boundaries.
4. Public or wildcard trust conditions.
5. Privilege escalation via misconfigured roles.
6. Lack of MFA or conditional access.
7. Untracked cross-account access.
8. No audit or monitoring of role usage.

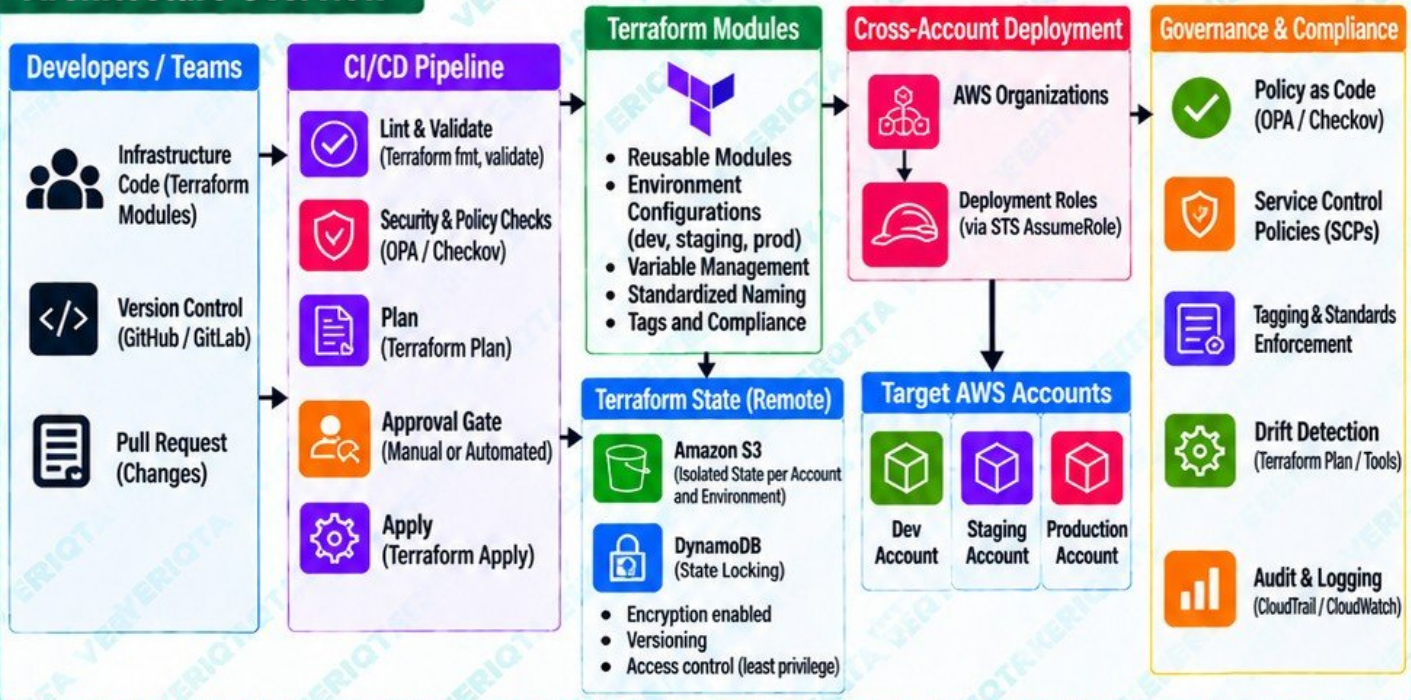
Objective

Build Terraform modules, isolated state, cross-account deployment roles, policy checks, drift detection, and approval gates.



Senior Challenge: Recover safely from partial Terraform failure without destructive replacement of stateful infrastructure.

Architecture Overview



Key AWS Services

Service	Purpose
Terraform	Infrastructure as code
Amazon S3	Remote state storage
Amazon DynamoDB	State locking
AWS IAM	Deployment roles and permissions
AWS STS	Cross-account role assumption
AWS Organizations	Multi-account management
AWS CodePipeline	CI/CD pipeline
AWS CodeBuild	Build and validate Terraform
AWS CodeDeploy	Controlled deployments (if needed)
AWS CloudTrail	Audit and API activity logs
AWS Config	Detect configuration drift
AWS Security Hub	Compliance and security findings
Amazon CloudWatch	Monitoring and alarms

Infrastructure Delivery Flow

- 1 Developer creates or updates Terraform code.
- 2 Code is pushed to Git repository.
- 3 CI pipeline runs validation, security checks, and Terraform plan.
- 4 Changes go through approval gate.
- 5 Pipeline applies changes using deployment role (STS).
- 6 State is stored in S3 with DynamoDB locking.
- 7 Drift detection runs regularly.
- 8 All actions are logged in CloudTrail and monitored in CloudWatch.

Best Practices



- Use modular and reusable Terraform code.
- Keep state in S3 with DynamoDB locking.
- Use least privilege IAM roles.
- Enforce policy checks before apply.
- Implement approval gates for production.
- Tag all resources consistently.
- Monitor and alert on drift.
- Use separate state per environment/account.
- Enable CloudTrail and Config.

Common Failure Scenarios



1. Partial Terraform apply (some resources created, others failed).
2. Accidental destructive changes to stateful resources.
3. State file corruption or accidental deletion.
4. Insufficient permissions for deployment role.
5. Policy check failures blocking deployment.
6. Configuration drift not detected.
7. Cross-account trust misconfiguration.
8. Concurrent state updates causing lock issues.

Recovery Strategies



1. Use terraform state list and terraform state show to inspect current state.
2. Manually import existing resources (terraform import) if needed.
3. Use targeted apply (terraform apply -target) for safe recovery.
4. Avoid full destroy/replace of stateful resources.
5. Fix failed resources and re-run apply.
6. Validate in non-production first.
7. Review logs and CloudTrail for what changed.
8. Implement stronger plan review and approval process.

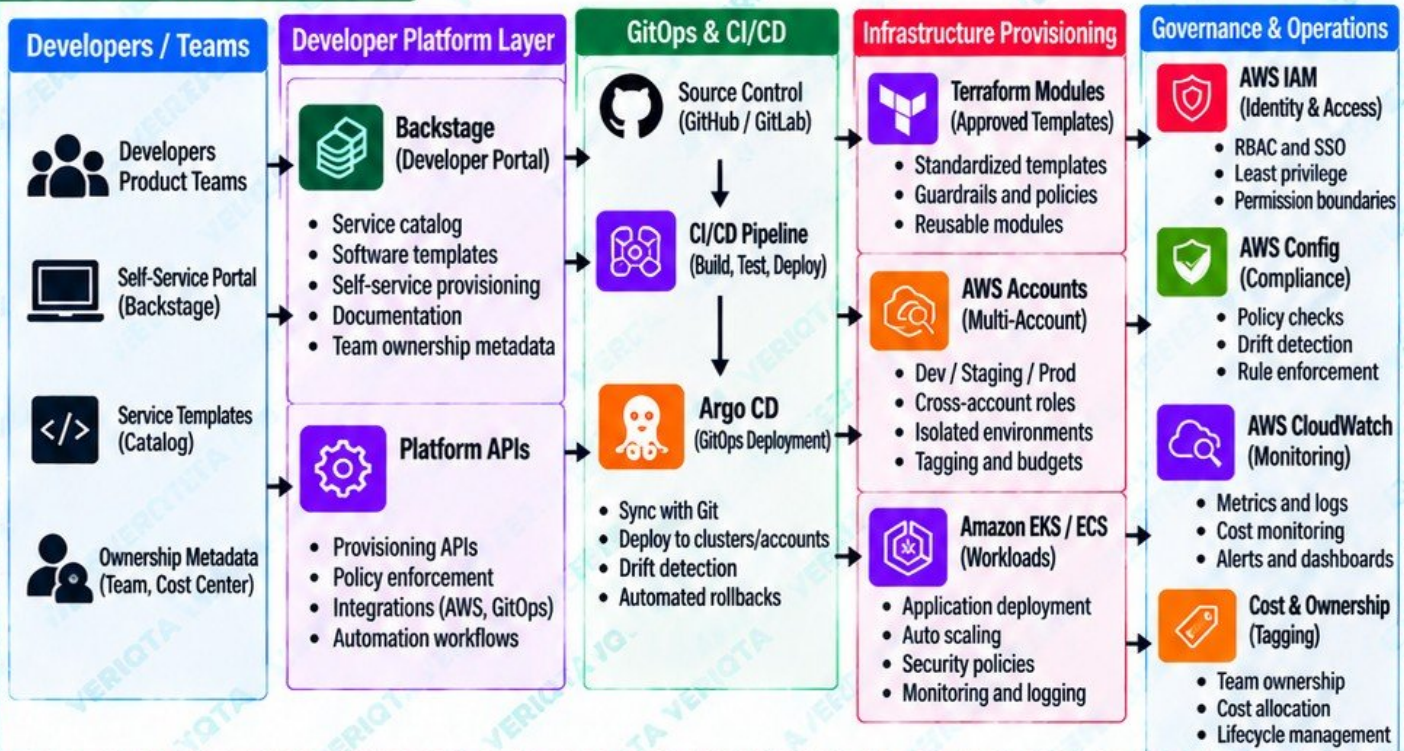
Objective

Create self-service workload provisioning with Backstage, platform APIs, GitOps, approved infrastructure templates, and ownership metadata.



Senior Challenge: Provide developer autonomy while enforcing tenancy, security policies, lifecycle ownership, and cost accountability.

Architecture Overview



Key AWS Services

Service	Purpose
 Backstage	Developer portal and service catalog
 AWS Organizations	Multi-account environment
 AWS IAM Identity Center	SSO and access management
 AWS STS	Cross-account role assumption
 AWS EKS / ECS	Application workloads
 AWS CodePipeline	CI/CD automation
 Argo CD	GitOps deployment
 Terraform	Infrastructure as code
 AWS Config	Compliance and drift detection
 AWS CloudWatch	Monitoring and alerting
 AWS Service Catalog	Approved templates (optional)
 AWS Cost Explorer	Cost tracking and accountability

Developer Self-Service Flow

- 1 Developer selects a template in Backstage.
- 2 Provides parameters (name, environment, size).
- 3 Platform APIs validate policies and quotas.
- 4 Approved template triggers CI/CD pipeline.
- 5 Terraform provisions infrastructure in target account.
- 6 Argo CD deploys application (GitOps).
- 7 Ownership metadata and tags are applied.
- 8 Monitoring, cost tracking, and compliance are enabled.

Best Practices

- ✓ Use Backstage with standardized templates.
- ✓ Enforce tenancy with separate accounts.
- ✓ Apply IAM Identity Center (SSO) and RBAC.
- ✓ Use approved Terraform modules.
- ✓ Implement policy checks (OPA, Checkov).
- ✓ Tag all resources with ownership metadata.
- ✓ Enable cost allocation and budgets.
- ✓ Use GitOps for continuous deployment.
- ✓ Monitor compliance with AWS Config.
- ✓ Provide clear documentation and runbooks.

Common Failure Scenarios



1. Developers bypassing the platform (shadow IT).
2. Over-permissive IAM roles and policies.
3. Missing or incorrect resource tagging.
4. Cost overruns due to lack of quotas and budgets.
5. Misconfigured templates causing insecure deployments.
6. Drift between actual infrastructure and code.
7. Lack of lifecycle ownership (orphaned resources).
8. Non-compliance with organizational policies.

Senior Challenge Considerations



1. Provide developer autonomy without losing control.
2. Enforce security policies and guardrails automatically.
3. Ensure tenancy and isolated environments.
4. Track lifecycle ownership and cost accountability.
5. Prevent privilege escalation through platform templates.
6. Detect and remediate drift.
7. Handle multi-account and multi-region deployments.
8. Scale the platform for multiple teams and services.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy

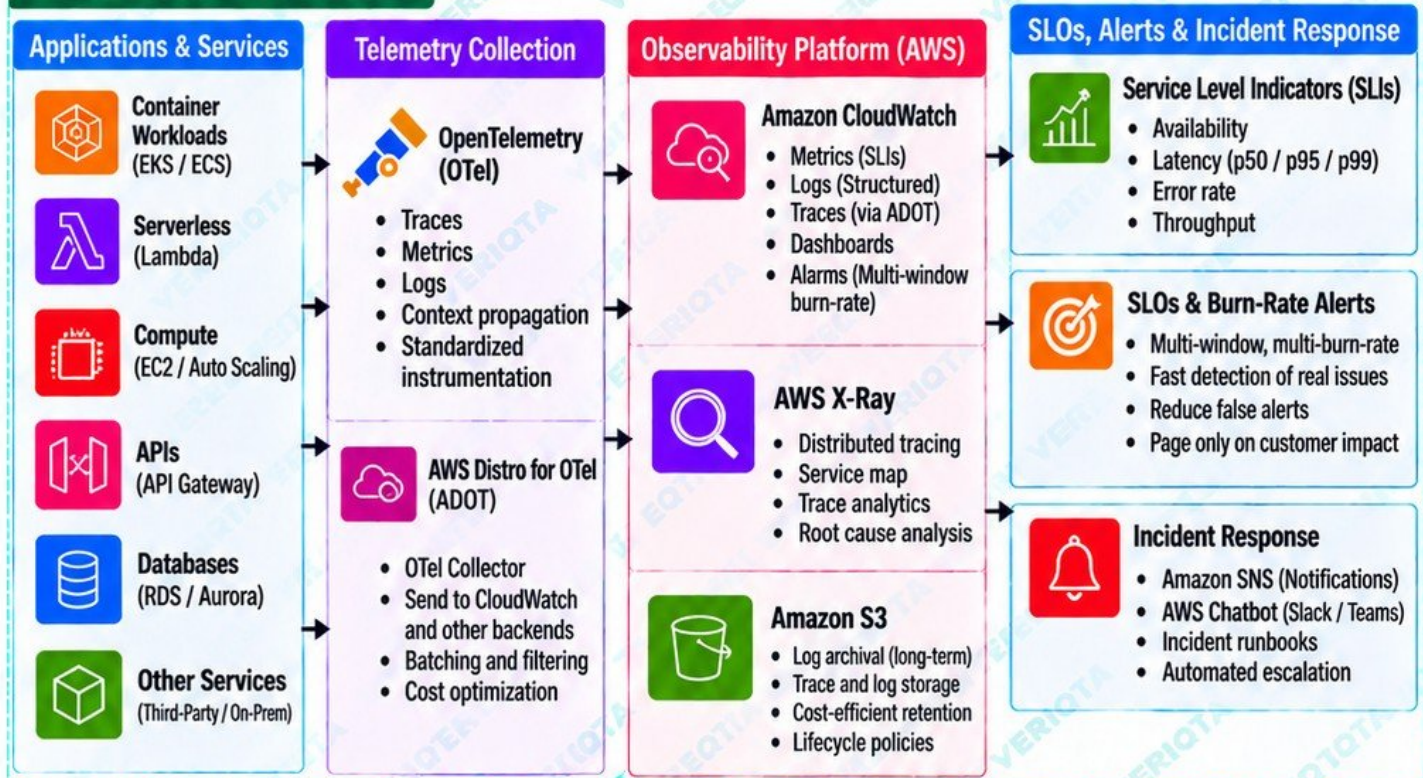
Objective

Integrate CloudWatch, OpenTelemetry, traces, structured logs, service-level indicators, and multi-window burn-rate alerts.



Senior Challenge: Detect customer-impacting degradation while reducing noisy alerts and controlling telemetry cost.

Architecture Overview



Key AWS Services

Service	Purpose
 Amazon CloudWatch	Metrics, logs, dashboards, alarms
 AWS X-Ray	Distributed tracing and service map
 AWS Distro for OpenTelemetry	Telemetry collection and export
 Amazon S3	Long-term log and trace storage
 AWS Lambda	Log processing and automation
 Amazon SNS	Alert notifications
 AWS Chatbot	Slack/Teams integration
 Amazon Managed Prometheus	Metrics for Prometheus
 Amazon Managed Grafana	Dashboards and visualization
 AWS IAM	Access control and least privilege

Observability Flow

- 1 Instrument applications with OpenTelemetry.
- 2 Collect traces, metrics and logs using ADOT.
- 3 Send telemetry to CloudWatch and X-Ray.
- 4 Define SLIs and SLOs for critical services.
- 5 Create multi-window burn-rate alerts.
- 6 Route alerts via SNS and Chatbot.
- 7 Investigate using traces, logs and dashboards.
- 8 Take action using runbooks and automate remediation.

Best Practices

- Instrument all services with OTel.
- Use structured, consistent log format.
- Define SLIs/SLOs per service.
- Use multi-window burn-rate alerts.
- Set meaningful thresholds (avoid noise).
- Implement log sampling and retention.
- Control telemetry cost with filters and exclusion rules.
- Use service maps and traces for faster RCA.
- Regularly review and tune alerts.
- Document runbooks and ownership.

Common Failure Scenarios

1. High latency not detected early enough.
2. Alert noise causing alert fatigue.
3. Missing or incorrect instrumentation.
4. Incomplete traces or broken context.
5. Excessive telemetry cost.
6. Misconfigured SLOs and thresholds.
7. Delayed incident response.
8. Logs not retained for investigation.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy

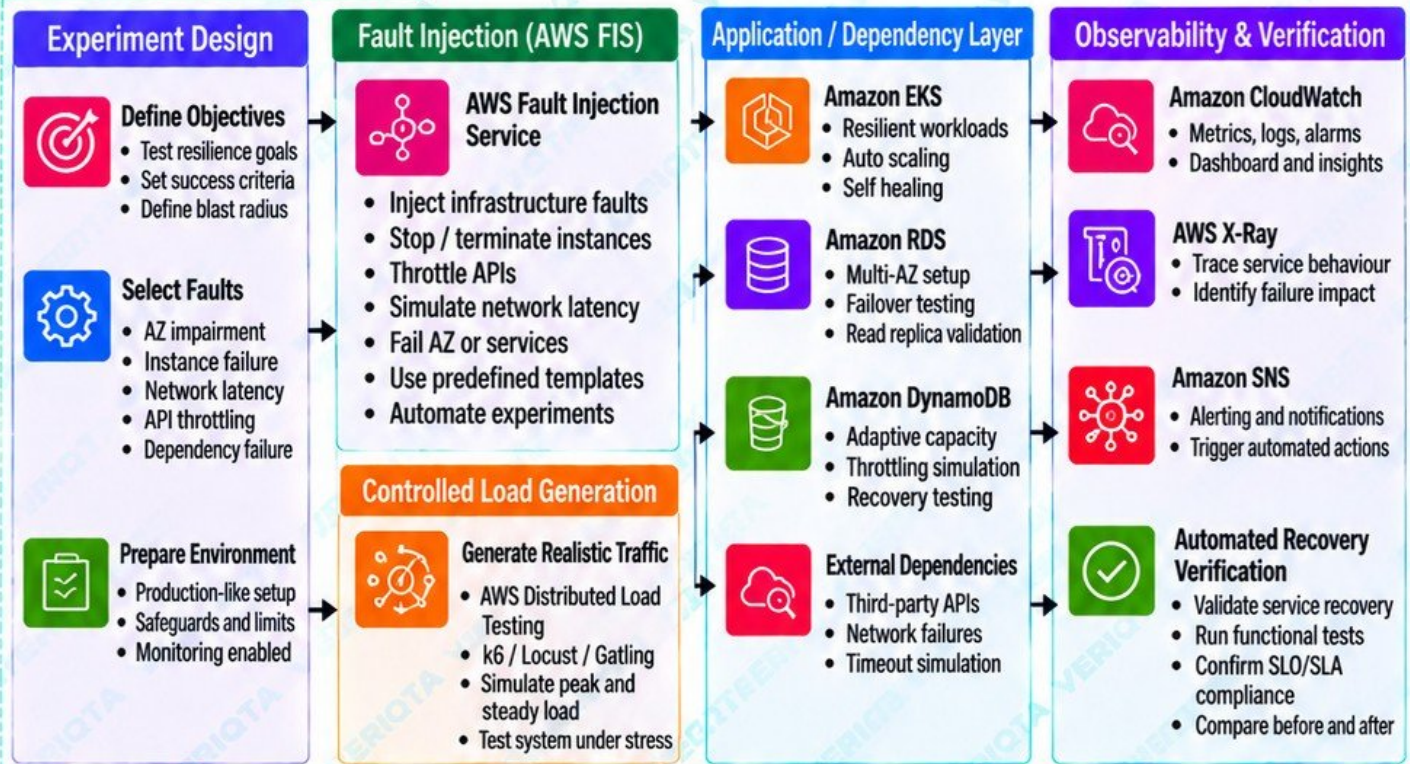
Objective

Use AWS Fault Injection Service, controlled load generation, dependency fault simulation, and automated recovery verification.



Senior Challenge: Validate recovery from AZ impairment, throttling, and dependency timeouts without exceeding an approved blast radius.

Architecture Overview



Key AWS Services

Service	Purpose
 AWS Fault Injection Service	Inject controlled faults
 AWS Distributed Load Testing	Generate realistic load
 Amazon CloudWatch	Metrics, logs and alarms
 AWS X-Ray	Tracing and dependency map
 Amazon EKS	Resilient application platform
 Amazon EC2	Instance failure testing
 Amazon RDS	Database failover and recovery
 Amazon DynamoDB	Throttling and resilience testing
 Amazon SNS	Alerting and notifications
 AWS Lambda	Automated recovery actions
 AWS Step Functions	Orchestrate test and validation
 AWS Systems Manager	Runbooks and automation

Resilience Testing Flow

- 1 Define test objectives and blast radius.
- 2 Configure monitoring, alarms and dashboards.
- 3 Generate baseline load and capture metrics.
- 4 Inject controlled faults using AWS FIS.
- 5 Observe impact on applications and dependencies.
- 6 Validate automated recovery mechanisms.
- 7 Run functional tests after recovery.
- 8 Confirm SLO/SLA compliance.
- 9 Review results and identify improvements.

Best Practices

- Start with small, controlled experiments.
- Define and enforce blast radius limits.
- Use realistic production-like load.
- Test both infrastructure and dependencies.
- Validate automated recovery.
- Monitor application-level and business metrics.
- Document results and runbooks.
- Run tests regularly (game days).
- Continuously improve based on findings.

Common Failure Scenarios

1. AZ impairment causing service loss.
2. API throttling from dependencies.
3. Database failover latency.
4. High error rates during traffic spikes.
5. Cascading failures across services.
6. Recovery taking longer than expected.
7. Exceeding blast radius and impacting other services.
8. Automated recovery not working as designed.



Objective

Engineer cost allocation, workload rightsizing, Savings Plans strategy, storage lifecycle policies, and unit-economics dashboards.



Senior Challenge: Reduce unnecessary spend without weakening redundancy, capacity headroom, or recovery guarantees.

Architecture Overview

Data Collection & Allocation



AWS Cost Explorer

- Cost and usage data
- Trend analysis



AWS CUR

- Detailed cost and usage
- Export to S3



AWS Cost Allocation Tags

- Environment, team, application, owner



AWS Organizations

- Multi-account cost aggregation



AWS Budgets

- Cost budgets and alerts
- Anomaly detection

Optimization & Rightsizing



Compute Optimization

- Amazon EC2 (rightsizing)
- Compute Optimizer
- Auto Scaling



Container Optimization

- Amazon EKS / ECS
- Right-size requests
- Use Spot Instances
- Cluster autoscaling



Database Optimization

- Amazon RDS
- Rightsizing
- Reserved capacity
- Multi-AZ where needed



Load Balancer Optimization

- ALB / NLB
- Remove unused
- Optimize routing

Savings & Purchase Strategy



Savings Plans

- Compute Savings Plans
- EC2 Instance Plans
- Analyze utilization
- Commitment strategy



Reserved Instances

- Long-term workloads
- Cost predictability
- Coverage analysis



Spot Instances

- Non-critical workloads
- Batch and async jobs
- Fault-tolerant services



Purchase Planning

- Usage forecasting
- Blended strategy
- Review and adjust

Storage Lifecycle & Data Management



Amazon S3 Lifecycle

- Intelligent-Tiering
- Transition to IA / Glacier
- Delete expired data



Amazon EBS

- Rightsize volumes
- Use gp3
- Delete unused volumes
- Snapshot lifecycle



Amazon EFS

- Lifecycle policies
- Infrequent Access
- Remove unused data



Amazon RDS

- Automated backups
- Snapshot retention
- Remove old snapshots

Cost Visibility Dashboard

Total Monthly Cost

\$12,480

↑ 8%

Projected Cost

\$11,200

↓ 10%

Savings Plan Coverage

78%

↑ 12%

Unused Resources

\$1,230

↓ 45%

Cost by Service



Cost by Environment



Unit Economics



Cost per Customer

- Total infrastructure cost
- Active customers
- Trend and forecast



Cost per Service

- Tag-based allocation
- Shared vs dedicated
- Identify top consumers



Business Metrics

- Cost per transaction
- Cost per request
- Cost per environment

Best Practices



- Use clear and consistent tagging.
- Rightsize resources regularly.
- Use Savings Plans for stable workloads.
- Use Spot Instances for non-critical workloads.
- Implement storage lifecycle policies.
- Remove unused and idle resources.
- Monitor with AWS Budgets and alerts.
- Track unit economics and business metrics.
- Maintain redundancy and capacity headroom.
- Review costs regularly with stakeholders.

Key AWS Services

Service	Purpose
AWS Cost Explorer	Analyze cost and usage
AWS CUR	Detailed cost and usage reports
AWS Budgets	Budgeting and cost alerts
AWS Organizations	Multi-account cost management
AWS Compute Optimizer	Rightsizing recommendations
AWS Savings Plans	Commitment-based discounts
Amazon EC2	Compute workloads
Amazon EKS / ECS	Container workloads
Amazon RDS	Managed databases
Amazon S3	Object storage and lifecycle
Amazon EBS	Block storage
AWS Trusted Advisor	Cost optimization recommendations

Optimization Flow

1. Collect and allocate costs using tags and CUR.
2. Analyze usage and identify rightsizing opportunities.
3. Implement Savings Plans and Reserved Instances.
4. Optimize storage with lifecycle policies.
5. Remove unused resources.
6. Monitor with budgets, dashboards, and unit economics.
7. Review regularly and maintain redundancy and performance.
8. Continuously optimize without impacting reliability.

Common Failure Scenarios



1. Over-provisioned and idle resources.
2. Missing or inconsistent tagging.
3. Low Savings Plan coverage.
4. Unused EBS and RDS snapshots.
5. Old data not moved to cheaper storage classes.
6. Spot Instance interruptions affecting critical workloads.
7. Cost optimization reducing redundancy or capacity headroom.
8. Lack of monitoring and budget alerts.



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy



@veriqta_academy

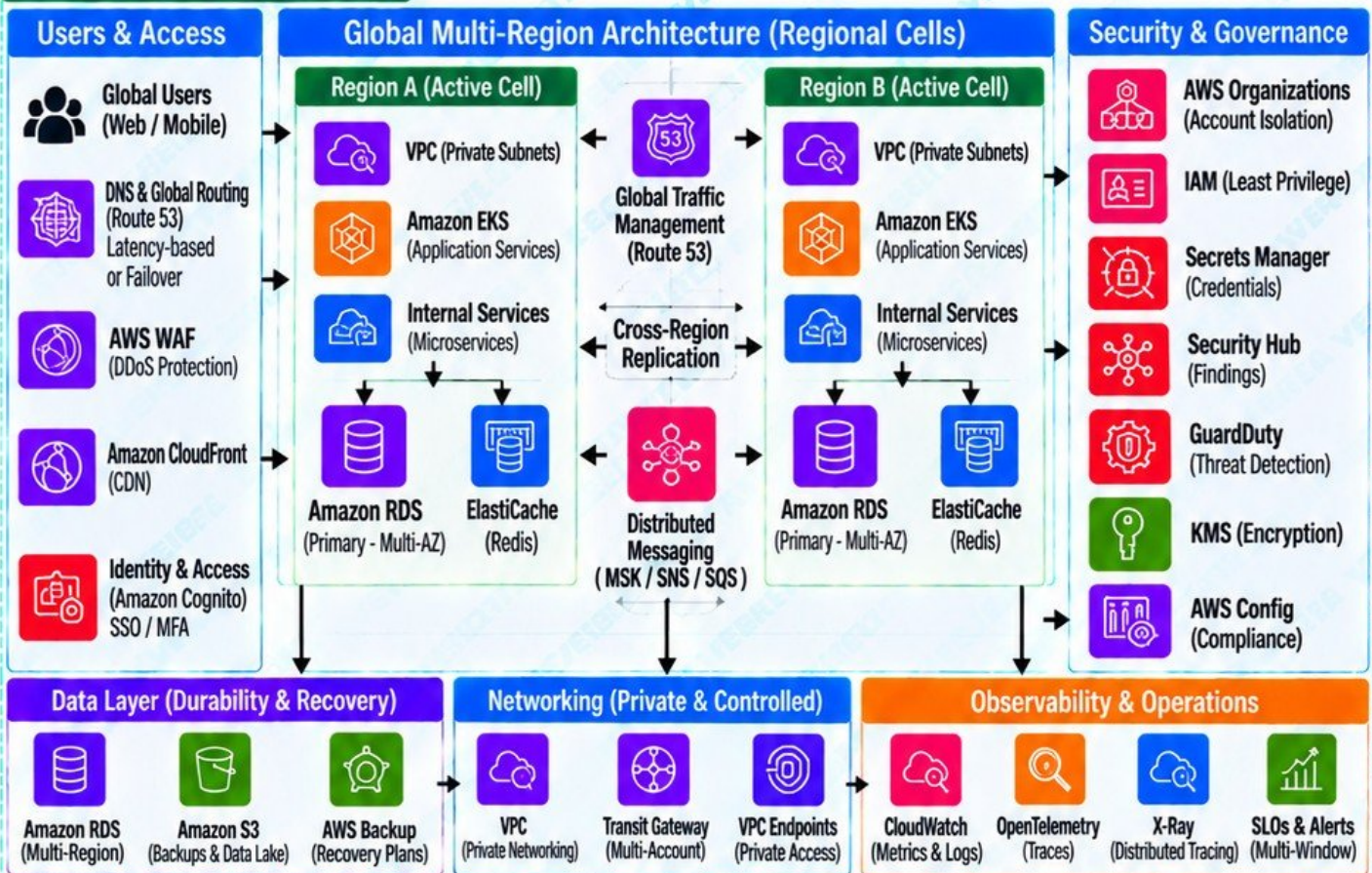
Objective

Combine account isolation, regional cells, private networking, EKS, distributed messaging, data recovery, security controls, and SLO-driven operations.



Senior Challenge: Defend the design against regional outage, compromised credentials, failed releases, traffic spikes, and cost constraints.

Architecture Overview



Key AWS Services

Service	Purpose
Route 53	Global routing and failover
CloudFront	CDN and edge caching
AWS WAF	DDoS and application protection
AWS Organizations	Account isolation and governance
Amazon EKS	Kubernetes for application workloads
Amazon RDS	Managed relational database
Amazon ElastiCache	Caching (Redis)
Amazon MSK / SQS / SNS	Distributed messaging
Amazon S3	Object storage and backups
AWS Backup	Data recovery and backup management
Transit Gateway	Multi-account and multi-region networking
AWS Secrets Manager	Credential management
GuardDuty	Threat detection
Security Hub	Security findings and compliance
CloudWatch / OpenTelemetry	Observability (metrics, logs, traces)

Design Flow

- 1 Define requirements (global, scalable, secure, resilient, cost-aware).
- 2 Set up account structure and governance (AWS Organizations).
- 3 Design networking (VPC, Transit Gateway, private endpoints).
- 4 Deploy EKS in multiple regions (regional cells).
- 5 Implement data layer (RDS, S3, backup and replication).
- 6 Add distributed messaging (MSK / SNS / SQS).
- 7 Enable security controls (IAM, KMS, GuardDuty, Security Hub).
- 8 Implement observability and SLOs (CloudWatch, OTel, X-Ray).
- 9 Test failure scenarios (region outage, credential compromise, release failure, traffic spikes).
- 10 Optimize cost and validate recovery.

Best Practices

- Use multi-region architecture with regional cells.
- Isolate accounts by environment and team.
- Use private networking and VPC endpoints.
- Implement least privilege access (IAM).
- Enable automated backups and tested recovery.
- Use SLOs, error budgets and multi-window alerts.
- Design for graceful degradation and failover.
- Implement cost controls and continuous optimization.
- Regularly test disaster recovery and incident response.
- Document architecture, runbooks and ownership.

Common Failure Scenarios



1. Regional outage causing service unavailability.
2. Compromised credentials and unauthorized access.
3. Failed deployment or release impacting users.
4. Sudden traffic spikes and resource exhaustion.
5. Data corruption or failed recovery.
6. Misconfigured security controls.
7. Networking or DNS failures.
8. Cost overruns due to unoptimized resources.

